Beyond Static Security: A Context-Aware and Real-Time Dynamic Zero Trust Architecture for IIoT Access Control

Fatemeh Stodt, *Student Member, IEEE*, Christoph Reich, and Fabrice Theoleyre, *Senior Member, IEEE*

***Abstract*—In industrial environments, cyber threats are escalating at an unprecedented rate, yet many existing security solutions fail to account for both contextual factors and the criticality of different network segments. This challenge is especially pronounced in diverse, large-scale, and highly dynamic Industrial Internet of Things (IIoT) environments. This article presents a dynamic zero trust access control (ZTA) model that adapts to real-time device status, network conditions, and user behavior to enforce context-aware, security-driven access decisions. At its core, our framework combines mathematical threat assessment with fuzzy logic-based state management (FSM) to continuously adjust trust levels and access permissions. We validated our approach through a Proof-of-Concept using a cluster of virtual machines (VMs) to simulate a controlled environment. This setup demonstrates the ZTA model's effectiveness in small-scale networks and provides a foundation for testing various access scenarios and evaluating security policies.**

***Index Terms*—Policy decision point (PDP), policy enforcement point (PEP), role-based access control (RBAC), zero trust architecture (ZTA).**

I. Introduction

IN CRITICAL infrastructures, such as manufacturing systems, cyber threats pose increasingly significant risks. Attacks like advanced persistent threats (APTs) and insider threats can severely disrupt operations, cause data breaches, and incur substantial operational and financial costs [1]. The rapid proliferation of Internet of Things (IoT) devices further exacerbates security challenges by expanding the attack surface, thus multiplying entry points available to attackers.

Traditional perimeter-based security models face limitations in complex industrial environments that integrate cloud services, remote operations, and diverse IoT devices. Such models cannot adequately mitigate threats involving lateral movement or exploitation of internal vulnerabilities. To address these challenges, zero trust architecture (ZTA) [2] has emerged as a robust security paradigm, characterized by its principle of continuous verification, where no user or device is implicitly trusted.

Static implementations of ZTA typically define fixed security policies, which lack the flexibility to adapt swiftly to dynamic conditions, such as changing device statuses, varying user behaviors, or evolving threats [3]. Consequently, static ZTA approaches cannot effectively respond to real-time threats or accommodate dynamic operational requirements Industrial Internet of Things (IIoT) environments.

To overcome these limitations, dynamic ZTA approaches have emerged, incorporating real-time contextual data to adapt security policies. Yao et al. [4] proposed a behavior-based trust model using fixed thresholds, while Feng and Hu [5] designed a multilayered control mechanism relying on periodic evaluations. While our current implementation also uses fixed thresholds, these are selected through empirical tuning and sensitivity analysis to ensure reliable performance across operational contexts. Unlike prior work, our architecture includes a modular threat assessment pipeline designed to support dynamic threshold adjustment in future iterations, enabling more adaptive and fine-grained policy enforcement over time.

In industrial contexts, the inherent heterogeneity of devices, stringent real-time operational demands, and varying criticality levels across network segments pose additional challenges. Feng and Hu [5] presented a multilayer dynamic access control mechanism, although their approach uses periodic rather than real-time evaluations, introducing potential security gaps.

Our proposed solution significantly advances the state-of-the-art by introducing a comprehensive, real-time adaptive ZTA model specifically tailored for IIoT environments. We utilize an integrated mechanism combining real-time contextual information, mathematical threat assessment, and a finite state machine (FSM)-based adaptive policy enforcement. Distinctively, our framework simultaneously considers contextual factors and operational criticality, thereby achieving granular, dynamic security policy adaptation tailored to industrial IoT scenarios.

The primary contributions of our work are summarized as follows.

1) A comprehensive context-aware access control mechanism integrating real-time device status, network

Received 26 March 2025; revised 12 May 2025 and 2 June 2025; accepted 5 June 2025. Date of publication 12 June 2025; date of current version 25 August 2025. This work was supported in part by the Federal Ministry of Education and Research supervised by Projektträger Karlsruhe under Grant COSMIC-X 02J21D144, and in part by the Region Grand Est., through the ENGLAB Project. *(Corresponding author: Fabrice Theoleyre.)*

Fatemeh Stodt is with the Institute for Data Science, Cloud Computing and IT Security, University of Applied Sciences Furtwangen, 78120 Furtwangen, Germany, and also with the ICube Laboratory, CNRS/University of Strasbourg, 67081 Strasbourg, France (e-mail: fatemeh.stodt@hs-furtwangen.de).

Christoph Reich is with the Institute for Data Science, Cloud Computing and IT Security, University of Applied Sciences Furtwangen, 78120 Furtwangen, Germany (e-mail: christoph.reich@hs-furtwangen.de).

Fabrice Theoleyre is with the ICube Laboratory, CNRS/University of Strasbourg, 67081 Strasbourg, France (e-mail: fabrice.theoleyre@cnrs.fr).

Digital Object Identifier 10.1109/JIOT.2025.3579028

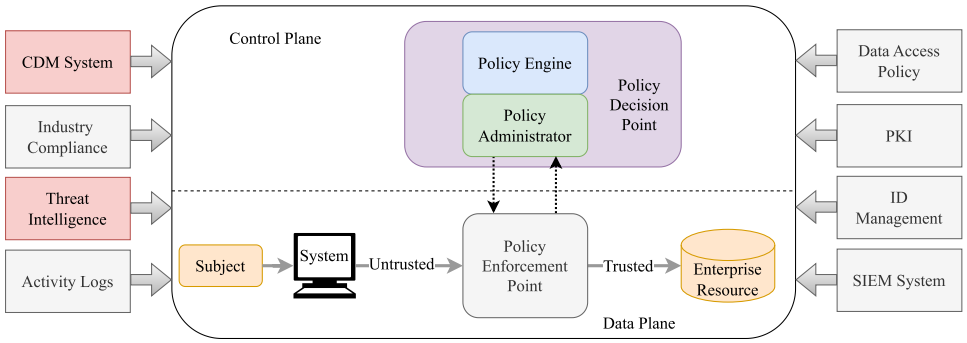

Fig. 1. Core zero trust logical components [6].

TABLE I
LIST OF ABBREVIATIONS USED THROUGHOUT THIS ARTICLE

Abbreviation	Definition
APT	Advanced Persistent Threat
CPU	Central Processing Unit
FSM	Finite State Machine
IIoT	Industrial Internet of Things
IoT	Internet of Things
PA	Policy Administration
PE	Policy Engine
PEP	Policy Enforcement Point
QoS	Quality of Service
RBAC	Role-Based Access Control
ZTA	Zero Trust Architecture

conditions, and user behavior to dynamically manage access permissions.

2) A real-time adaptive ZTA framework capable of immediate response to security incidents, significantly improving upon static or periodically evaluated ZTA systems.
3) A network segmentation strategy based on operational criticality, prioritizing security resources and efforts toward assets of highest importance.

The remainder of this article is structured as follows. Section II reviews related work and positions our approach within the existing literature. Section III details our proposed methodology, including FSM design and threat risk assessment mechanisms. Section VI describes our experimental setup and test scenarios, while Section VII provides quantitative evaluation results. Finally, Section VIII summarizes key insights and concludes this article. Table I lists the abbreviations used in this article.

II. BACKGROUND AND RELATED WORK

A. Zero Trust Architecture

ZTA was first introduced by Kindervag [7] and later expanded in NIST SP 800-207 [6]. Evan Gilman outlined five key principles: 1) the Internet is inherently insecure; 2) threats exist both internally and externally; 3) network-based trust is unreliable; 4) all entities must undergo strict authentication and authorization; and 5) access control must be dynamic, relying on continuous monitoring and adaptive authorization [8].

These core principles, such as "never trust, always verify," emphasize stringent security measures like micro-segmentation and least privilege access. They enforce strict access controls by dividing the network into smaller segments and ensuring that users and devices have only the minimal necessary access to perform their functions [9].

However, static Zero Trust implementations, while crucial in introducing these principles, have several limitations. Static policies struggle with inflexibility and administrative overhead, making them ill-suited for real-time threats and changes in device states [10]. Static, border-based security measures are ineffective against internal threats once attackers bypass the perimeter, as they offer minimal oversight within the network, which poses a critical vulnerability given the increasing mobility of devices and cloud-based operations [11]. Additionally, role-based access control (RBAC), often used with static policies, assigns permissions based on predefined roles, leading to coarse-grained access that can expose sensitive resources [12]. This static approach overlooks the need for continuous trust evaluation, which dynamically adjusts permissions based on user behavior, context, and risk [13].

The implementation of zero trust architecture (ZTA) is built upon a set of core logical components that collaboratively enforce strict access control and continuous verification. As illustrated in Fig. 1, these components include:

1) The policy engine (PE) implements access control based on organizational policies and contextual information.
2) The policy administration (PA) executes the decisions made by the PE, configuring paths for authorized sessions, granting and revoking access in real-time.
3) The policy enforcement points (PEP) is the gatekeeper enforcing access control decisions. It intercepts access requests and implements micro-segmentation and least privilege access

Together, these components enforce Zero Trust by replacing static perimeter defenses with dynamic, context-aware access control that adapt in real time to evolving risks.

B. Context

Static policies lack adaptability, failing to account for the dynamic nature of user behavior, device security posture, and network integrity [14]. With the rise of real-time threats and

constantly evolving network conditions, the need for dynamic adaptation in ZTA has become increasingly critical.

ZTSFC [15] introduces a new method for integrating monitoring and security (MaS) functions into ZTA. This integration allows policy decision points (PDPs) to make access decisions based on the trust level of each resource access request (RAR). ZTSFC extends typical ZTA models by steering traffic dynamically through a chain of service functions, enabling more precise security measures. SYSFLOW [16] extends Zero Trust principles with a system flow-based model. It abstracts system activities as flows, providing unified data and control plane separation. It allows developers to specify security intents, enforce micro-segmentation, dynamically assess risks, and respond to system events with reactive and proactive flow rule management. However, both SYSFLOW and ZTSFC assume that core components remain uncompromised.

Niu et al. [17] proposed an online verification mechanism for Zero Trust policies that uses logical abstraction and runtime data to validate compliance. While effective in leveraging system context, the approach depends on accurate predefined rules and may face limitations in handling complex or evolving attack scenarios. Yao et al. [4] proposed a trust-based access control (TBAC) model based on user behavior. However, it overlooks key network context and relies on static, nonreal-time policies with static threshold values, updated periodically.

eZTrust [18] introduces a perimeterization approach for securing microservices by using eBPF to trace events, assign context-based tags to packets, and verify them in real time. This enables precise, fine-grained security for interservice communication through fast and slow paths, depending on context. However, it relies on a trusted infrastructure provider, reducing its robustness in large-scale, dynamic environments.

C. ZTA for IIoT

Several recent efforts have focused on enhancing access control in industrial cyber physical systems (icps) through context-aware and adaptive mechanisms. Feng and Hu [5] proposed a Cyber–Physical ZTA featuring a multilayer control engine that evaluates trust scores for individual components while accounting for cross-layer interactions. An integrated policy optimizer, leveraging reinforcement learning and physical system models, dynamically refines access decisions to better align with the operational realities of icps. However, the framework's effectiveness relies heavily on the accuracy of real-time contextual data—a significant challenge in large-scale, complex systems.

Paul and Rao's Zero Trust model for smart manufacturing enhances IoT and cyber–physical system security through micro-segmentation and device discovery [19]. However, its effectiveness is limited by reliance on accurate device identification and assumptions about internal trust. Zanasi et al. [20] also explored micro-segmentation but focus on the network infrastructure. The Software Defined Network architecture represents a unified abstraction for policy enforcement. However, the synchronization of configurations in complex topologies remains challenging.

Xiao et al. [21] reviewed context-aware, risk-based access control models in Zero Trust Internet of Things (IoT) and IIoT systems, highlighting the use of real-time context, ABAC, RBAC, and fuzzy logic for dynamic policy adjustment. They note challenges, such as data accuracy, centralization bottlenecks, and integration complexity.

Software-defined networking (SDN) decouples the control and data planes, enabling dynamic network policy enforcement and facilitating micro-segmentation, which aligns naturally with ZTA principles [22]. In particular, the network can detect malicious behaviors and then adapt the configuration to counteract detected threats [23]. This programmability allows for granular security policies and fine-tuned micro-segmentation, directly supporting dynamic security mechanisms within IoT networks [24].

Quality-of-Service-aware software-defined IoT (QoS-SD-IoT) advances the capabilities of SD-IoT by embedding Quality-of-Service (QoS) considerations directly into network management. It enables dynamic prioritization of critical IoT traffic while ensuring that security enforcement mechanisms do not compromise essential real-time operations [25]. This functionality is especially crucial in IIoT environments, where maintaining reliable QoS for mission-critical tasks is essential. Recent studies have advanced security in SDN-enabled IoT systems by incorporating deep learning-based cyberattack detection methods, leveraging SDN's programmability and centralized control for real-time anomaly detection and mitigation [26].

Moreover, specialized adaptations of SDN have emerged in distinct IoT domains. The Software-Defined Internet of Multimedia Things (SD-IoMT) specifically targets media streaming scenarios, optimizing data flow management and implementing adaptive, context-aware security measures for multimedia applications [27]. Similarly, the Software-Defined Internet of Vehicles (SD-IoV) emphasizes adaptive streaming and real-time policy enforcement tailored for latency-sensitive vehicular networks and intelligent transportation systems, addressing unique operational challenges, such as high mobility and stringent QoS requirements [28].

These mechanisms support the dynamic, context-aware security policies central to our ZTA model.

III. Proposed Architecture

Compared to previous works, our framework distinguishes itself by simultaneously addressing contextual information and the criticality of different network segments. Existing solutions typically focus on isolated aspects, such as user behavior or device compliance, without fully capturing their dynamic interplay. By integrating real-time threat assessment with continuous policy adaptation, our approach enables a more granular and proactive security model, making it highly scalable and efficient for diverse and rapidly evolving IIoT environments.

Fig. 2 illustrates our framework with a collection of collaborative components. This architecture ensures that every access request is carefully considered and managed dynamically

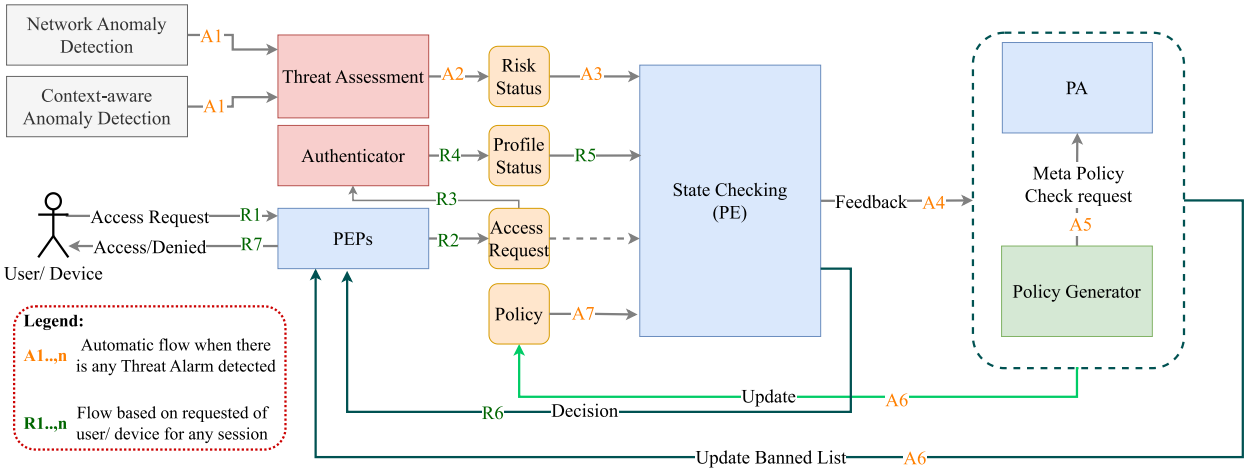

Fig. 2. Integrated framework for dynamic ZTA.

based on the context of the environment. We have designed our solution to provide the following characteristics.

1) *Dynamic Trust Adjustments:* Trust within the network is dynamic and varies based on context, device compliance, and user behavior. The architecture continuously assesses risk levels to adjust access permissions in real-time, ensuring security responses are aligned with current conditions.
2) *Continuous Policy Adaptation:* Because static security policies are inadequate in a dynamic IIoT environment, our policy generator continuously updates policies based on real-time assessments, ensuring the architecture remains responsive to emerging risks.
3) *Core Infrastructure Integrity Assumption:* The architecture assumes core components are initially uncompromised, enabling reliable anomaly detection. Any deviations are flagged, allowing the policy engine (PE) to adjust access permissions proactively.

A. Identificators

The operational workflow begins when an access request is made by an entity (such as a user, device, or application) seeking to access a network resource. Each entity is assigned a unique identifier, referred to as an *ID*, which the system uses to manage and authenticate access permissions. In this architecture,

1) *User ID* identifies an individual or a group.
2) *Device ID* identifies a device within the network.
3) *Flow ID* Flow ID refers to a unique identifier for a network communication session, typically derived from attributes, such as source IP address, destination IP address, source port, destination port, and protocol (5-tuple). It identifies a data flow or session, especially useful for monitoring continuous data streams.

IDs are generated and distributed centrally by the policy administration (PA) module during device onboarding or user registration. Device IDs are automatically distributed upon device enrollment, user IDs are provisioned during account creation, and flow IDs are generated dynamically per communication session by network monitoring components, then shared with the PEP and PE for consistent reference throughout access control processes.

The *ID* plays a central role in the access control workflow. The PEP verifies the entity's ID against a list of authorized or restricted entities when it receives an access request. In that way, only approved IDs can proceed to the authentication stage. The *Authenticator* module then validates the entity's credentials, using the ID to confirm its identity and integrity. Verified IDs are subsequently evaluated by the PE based on current security policies, threat assessments, and contextual information to determine whether access should be granted or denied.

Throughout the access request process, the *ID* serves as a consistent reference for tracking, authentication, and decision-making, enabling the system to enforce Zero Trust principles by verifying each unique user, device, and data flow within the network.

B. Policy Enforcement Points

PEPs are strategically positioned gatekeepers that manage access at critical network intersections [3]. The PEP plays an important role in processing access requests from various entities. The PEP holds a list of banned IDs provided by the PA. When an access request is received, the PEP checks the requesting entity's ID against this list. If the ID is not present in the banned list, the PEP forwards the entity ID to the Authenticator and the PE. After receiving a decision from the PE, which determines whether to grant or deny access, the PEP completes the access process and logs all attempts and outcomes for audit purposes.

The banned ID list is centrally maintained by the PA and synchronized periodically or triggered by security events across all PEPs. IDs can be removed (un-banned) either through administrative intervention after a reassessment of risk or automatically when anomaly detection indicates normal behavior has resumed. The list is synchronized frequently—typically event-driven—to ensure consistency across all PEPs.

C. Authenticator

The authenticator validates the identity of users or devices by checking credentials, certificates, and signatures to ensure authenticity. The resulting profile status (valid or not valid) is essential for the PE's decision-making and depends on factors, such as certificate validation time, installer signature verification, and approval signature confirmation.

D. Anomaly Detection

Network Anomaly Detection monitors network traffic to detect deviations from established patterns indicating possibly a threat.

The objective is to create an internal representation of network communications, serving as its *signature*. If the observed traffic pattern deviates significantly from this signature, the system detects and flags it as an anomaly. For this component, we rely on our previous work [29].

We also re-exploit here our context-aware anomaly detection method described in [30]. Additionally to the network traffic, we also consider context information, such as time, location, and user activities.

E. Policy Generator

This component updates or creates new policies based on findings from the Threat Assessment module and the security status of entities within the system, ensuring an adaptable security posture.

The sum of these interactions guides the decision of the PE to grant or deny access. If the decision is to deny access, the PEP enforces it.

F. Policy Engine

The PE evaluates access requests based on real-time security data and adaptive measures. It relies on a finite state machine (FSM) to manage and transition between logical states representing the risk levels of entities. This integration provides a structured approach to handling the dynamic nature of security management, enabling the PE to respond effectively to changes in an entity's behavior.

Within the PE, the FSM (Fig. 3) defines states which each corresponding to a specific security posture. Entities transition between these states based on triggers generated by real-time threat assessments and contextual analysis. FSM states include Normal, Alert, High Risk, Quarantined, and Compromised, transitioning based on explicit triggers with specific security implications.

1) *Normal to Alert:* Triggered by minor anomalies (Threat Risk score between 0.4–0.6), the Threat Risk Score is a quantitative metric aggregating normalized contextual criteria to assess the current security risk associated with a device, user, or network segment., prompting increased monitoring.
2) *Alert to High Risk:* Significant anomalies detected (Threat Risk between 0.6–0.8), triggering heightened security measures and restricted access.
3) *High Risk to Quarantined:* Critical threats detected (Threat Risk $\geq$ 0.8), enforcing isolation and stringent access limitations.
4) *Alert/High Risk/Quarantined to Compromised:* Confirmed security breaches via investigation, mandating immediate isolation and remediation.
5) *Compromised to Recovery:* Initiation of recovery procedures post-investigation to restore secure operations.
6) *Recovery to Normal:* Post-recovery verification and return to standard operational status upon confirmed resolution.

For instance, detecting privilege escalation with a high confidence level prompts the PE to transition the entity to a High Risk or Quarantined state, invoking stricter access measures or isolation protocols. This structured state management enables the PE to apply policy-driven actions tailored to each risk level, ensuring security responses are proportionate and timely.

The FSM includes a feedback loop, explicitly involving continuous monitoring and dynamic updates to authorized or banned entity IDs. If anomalous behavior is detected, entity IDs may be flagged and transitions between states occur accordingly, resulting in updates to the banned ID lists maintained by the PEP and PA. Conversely, when an entity previously marked as risky demonstrates normal behavior, it transitions to a lower-risk state, potentially resulting in its ID being removed from banned lists. This dynamic interplay ensures robust security by continuously aligning the access control measures with real-time assessments and evolving contextual information.

The low computational complexity of employing a FSM further supports real-time assessment and effective scalability in dynamic environments.

G. Threat Assessment

This module evaluates the potential risks associated with each access request within the ZTA. This module synthesizes inputs from Network and Context-aware Anomaly Detection to derive an overall threat risk score. The assessment employs a structured approach, incorporating statistical analysis and machine learning models to classify threats and assess their severity. This comprehensive analysis informs the PE and contributes to adaptive, context-aware decision-making.

H. Policy Administration

The PA component plays a central role in the proposed ZTA framework, serving as the authority for managing and maintaining security policies that govern access control across the network. It ensures that these policies are effectively deployed and dynamically updated to respond to an evolving security environment. Key responsibilities of the PA include the creation and management of policies, as well as the enforcement of meta-policies that adapt to changing contextual factors. A meta-policy, in this context, refers to a higher-level template that governs the creation and dynamic adjustment of specific access control rules.

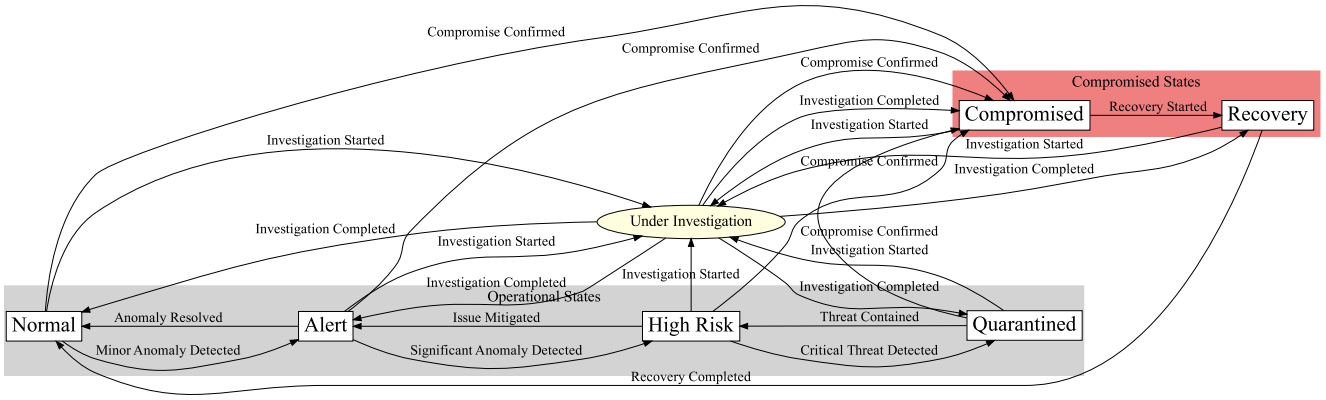

Fig. 3. FSM for PE.

TABLE II
OVERVIEW OF VARIABLES FOR THREAT RISK EVALUATION

Symbol	Description
C	Confidence of Threat
A	Attack Criticality
S	Segment Criticality
P	Past Anomalies
C_{norm}	Normalized Confidence of Threat
A_{norm}	Normalized Attack Criticality
S_{norm}	Normalized Segment Criticality
P_{norm}	Normalized Past Anomalies
w_C	Weight for Confidence of Threat
w_A	Weight for Attack Criticality
w_S	Weight for Segment Criticality
w_P	Weight for Past Anomalies

IV. THREAT RISK

Threat Assessment relies on a *Threat Risk* score calculated by combining four key criteria.

1) *Confidence of Threat* (C) represents the model's confidence level in identifying potential threats based on real-time data.
2) *Attack Criticality* (A) is a fixed value assigned based on the severity and potential impact of the detected attack type.
3) *Segment Criticality* (S) represents the importance or sensitivity of the targeted network segment.
4) *Past Anomalies* (P) reflects the historical frequency and severity of anomalies associated with the entity (e.g., user or device).

Each criterion is computed and normalized to ensure consistency across different scales before being combined into the overall Threat Risk score. Table II summarizes the notations used throughout the paper.

A. Confidence of Threat (C)

C is derived from real-time analysis using machine learning models or statistical methods that detect anomalies or malicious activities. For example, an intrusion detection system may assign a confidence score between 0 and 100% indicating the likelihood that observed behavior is malicious. We apply a Min-0Max Scaling for normalization

$$\text{Var}_{\text{norm}} = \frac{Var - \text{Var}_{\text{min}}}{\text{Var}_{\text{max}} - \text{Var}_{\text{min}}}$$

where Var_{min} and Var_{max} are the minimum and maximum possible confidence scores (typically 0 and 100) for a given variable Var, and Var_{norm} is its normalized value.

Min–Max Scaling is the most relevant here because the scores have a fixed range and need to be mapped consistently to [0,1]. This approach preserves the proportionality and interpretability of the data, which is critical for combining multiple criteria in the risk assessment. Alternative methods, like Z-score normalization, are less suitable as they distort the original scale.

B. Attack Criticality (A)

A is assigned based on the severity of the detected attack type, using standardized threat intelligence sources like the MITRE ATT&CK framework [31]. Each attack type is mapped to a criticality score reflecting its potential impact. Similar to C, A is normalized using Min–Max Scaling.

C. Segment Criticality (S)

S represents the importance of the network segment being accessed. Segments are assigned criticality scores based on factors like data sensitivity and business impact. Examples of criticality scoring include assigning higher scores to segments containing sensitive operational data (e.g., production control systems) or those directly impacting safety and business continuity. Prior works, such as [20] and [21] provide methods for quantifying segment criticality. We still employ Min–Max Scaling for normalization.

D. Past Anomalies (P)

P quantifies the historical frequency and severity of anomalies linked to the entity requesting access. This includes prior incidents and behavioral deviations.

A logarithmic transformation is applied to manage skewness due to outliers

$$P_{\log} = \log(P + 1)$$

followed by robust scaling:

$$P_{\text{norm}} = \frac{P_{\log} - \text{Median}(P_{\log})}{\text{IQR}(P_{\log})}$$

where Median($P_{\log}$) is the median and IQR($P_{\log}$) is the interquartile range of the transformed data.

E. Threat Risk Calculation and Categorization

The risk assessment process combines anomaly scores obtained from both network anomaly detection and context-aware anomaly detection modules. We compute a unified threat risk score using a weighted linear combination method, ensuring a clear and interpretable representation of the risk level for each entity

$$\text{Threat Risk} = \sum_{v \in \{C,S,A,P\}} w_v \cdot v_{\text{norm}}, \quad \text{where } \sum w_v = 1. \quad (1)$$

Here, the weights w_C, w_A, w_S, and w_P represent the relative significance assigned to the criteria: Confidence of Threat (C), Attack Criticality (A), Segment Criticality (S), and Past Anomalies (P), respectively.

We selected this approach due to its inherent transparency and interpretability, which allows stakeholders to clearly understand the contributions of each risk factor. The weighted combination model offers flexibility to dynamically adjust risk criteria based on evolving threat landscapes and operational priorities within IIoT environments.

Initial weights are expected to be configured by cybersecurity experts or system administrators who possess contextual awareness of the industrial network's operational priorities and the criticality of individual segments. Their domain-specific insight ensures that weighting reflects the relative importance of each risk factor in a real-world deployment.

In our experimental setup, these weights were manually assigned based on predefined assumptions and simulated criticality of entities within the testbed. To reflect realistic prioritization, we employed a contextual risk sensitivity tuning strategy. For example, a simulated sensor managing core pressure in a real-time production machine was treated as highly sensitive due to its immediate safety and operational impact. Conversely, a robotic transporter operating in a non-critical post-assembly stage (e.g., moving finished parts) was considered less sensitive, as it could be temporarily replaced by manual labor without compromising system safety or integrity.

This prioritization was guided by domain-driven assessments rather than statistical tuning, ensuring that the calculated Threat Risk score reflected not only the presence of anomalies but also the potential operational consequences of those anomalies. Currently, weights (wC, wA, wS, wP) are manually configured based on domain expertise, which provides clear interpretability but inherently limits adaptability and introduces subjectivity. To strengthen robustness, future work will integrate dynamic weight adjustment through machine learning and optimization methods. Approaches, such as supervised learning or reinforcement learning could optimize these weights based on empirical threat detection performance, while sensitivity analysis will be used to evaluate how variations in these weights influence the Threat Risk score.

Following the calculation, the Threat Risk score directly informs the entity's risk categorization, defining specific thresholds that translate numerical scores into actionable risk levels. Algorithm 1 illustrates the categorization procedure clearly:

Algorithm 1 Risk Level Determination

```
procedure DETERMINERISKLEVEL(EntityID, ThreatRisk)
    Input: EntityID, ThreatRisk (calculated from anomaly detection)
    Output: Updated risk level for EntityID
    if ThreatRisk ≥ 0.8 then
        UpdateRiskLevel(EntityID, "Critical Risk")
    else if ThreatRisk ≥ 0.6 then
        UpdateRiskLevel(EntityID, "High Risk")
    else if ThreatRisk ≥ 0.4 then
        UpdateRiskLevel(EntityID, "Low Risk")
    else
        UpdateRiskLevel(EntityID, "Normal Risk")
    end if
    Return CurrentRiskLevel(EntityID)
end procedure
```

Risk levels are continuously updated based on real-time data inputs, ensuring timely responsiveness to threat dynamics.

1) *Critical Risk ($\geq$ 0.8):* Triggers immediate restrictive actions due to high threat confidence.
2) *High Risk ($0.6 \leq score < 0.8$):* Activates heightened monitoring and stricter security controls.
3) *Low Risk ($0.4 \leq score < 0.6$):* Results in increased vigilance while preserving operational flexibility.
4) *Normal Risk (< 0.4):* Indicates standard operating conditions without additional restrictions.

The defined thresholds are based on empirical evaluations and industry-standard cybersecurity practices, designed to effectively balance robust security measures and operational performance. Organizations may adapt these thresholds to their specific operational requirements and security policies, allowing customized responsiveness to their particular security landscape.

V. Integration and Decision Process

The policy engine (PE) integrates a finite state machine (FSM) to manage dynamic transitions between security states based on event-driven triggers. The FSM operates by processing events, such as minor_anomaly_detected, significant_issue_detected, and anomaly_resolved, which correspond to real-time security observations. These events dictate transitions between states, such as *Normal*, *Alert*, *High Risk*, and *Quarantined*, allowing the system to respond adaptively to changing conditions.

The FSM is defined as a tuple (S, Σ, δ, s_0).

1) S: A finite set of states: *Normal*, *Alert*, *High Risk*, *Quarantined*, *Compromised*, *Recovery*.
2) Σ: A finite set of events representing observed anomalies or resolutions.
3) $\delta : S \times \Sigma \rightarrow S$: A state transition function mapping the current state and event to a new state.
4) s_0: The initial state, *Normal*.

An event-driven design empowers the PE to respond swiftly and effectively to real-time threats. Although the Threat Risk score influences event generation, the FSM transitions

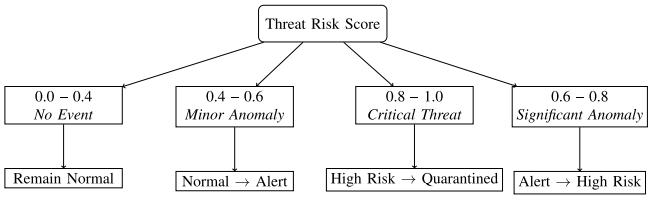

Fig. 4. Threat risk score mapping to FSM events.

deterministically based on these events, ensuring consistent and predictable state changes.

Fig. 3 illustrates the FSM. In particular, when a minor anomaly is detected, the FSM transitions from a *normal* state to an *Alert* state. On the contrary, a critical threat forces the system to go in quarantine, where isolation measures are enforced. The system returns to a normal state when anomalies are resolved.

A. Finite State Machine Event Generation

The FSM transitions between states based on events that are directly tied to the calculated Threat Risk scores. Events, such as *minor_anomaly_detected*, *significant_anomaly_detected*, or *critical_threat_detected* are triggered when the Threat Risk score surpasses predefined thresholds, determined as follows.

1) *Minor Anomaly Detected:* Triggered when the Threat Risk score is ≥ 0.4 and < 0.6.
2) *Significant Anomaly Detected:* Triggered when the Threat Risk score is ≥ 0.6 and < 0.8.
3) *Critical Threat Detected:* Triggered when the Threat Risk score is ≥ 0.8.

Initial thresholds (0.4, 0.6, and 0.8) were established manually based on practical judgment, influenced by industry-recognized best practices for anomaly severity classification and security response escalation. Specifically, guidance from frameworks, such as the *NIST SP 800* series [32] and the *OWASP Risk Rating Methodology* provided conceptual benchmarks [33] for defining progressive threat levels from minor anomalies to critical threats requiring isolation.

While these thresholds were not derived from formal optimization or machine learning, they were chosen to reflect common principles in risk management, such as prioritizing availability in safety-critical IIoT environments and avoiding over-sensitization. Within our experimental setup, we qualitatively validated that these thresholds produced FSM transitions aligned with expected security responses under simulated benign and malicious behaviors.

We acknowledge that this manual, rule-based calibration is a simplification. Future work will explore data-driven threshold refinement using operational telemetry and feedback loops [34] to improve precision and reduce false positives in dynamic conditions.

As illustrated in Fig. 4, each Threat Risk interval refers to a distinct event, which drives FSM state transitions.

B. Policy Creation and Management

The policy administration (PA) facilitates the creation, modification, and storage of security policies for various devices and entities within the network. Each policy includes conditions that reference the state of the entity as defined by the PE (*Normal*, *Alert*, *High Risk*, *Quarantined*, *Compromised*, and *Recovery*).

For example, a policy rule may specify.

1) *Permit* access to resource R if the entity state is *Normal* and the authentication level is sufficient.
2) *Deny* access to sensitive resource S if the entity state is *High Risk* or higher.
3) *Require* additional verification steps if the entity state is *Alert*.

These policies are stored in a centralized repository managed by the PA and are retrieved by the PE during the access control decision process.

C. Meta-Policy Enforcement and Validation

A distinctive feature of the policy administration (PA) is its enforcement of meta-policies, which are high-level rules governing the formulation and validation of new or updated policies. These meta-policies ensure that all policies adhere to organizational standards. The validation process considers both semantic correctness and risk alignment.

Policy validation involves the following steps.

1) *Semantic Analysis:* The PA verifies that policies align with high-level objectives and do not conflict with existing rules, ensuring their logical consistency. Meta-policies are precompiled into a decision tree structure, where each node represents a condition or constraint. The proposed policy is then abstracted into a feature vector and traverses the decision tree for validation. Conflicts are resolved based on predefined priority levels of meta-policies.
2) *Risk Alignment:* Policies are validated against the organization's risk management principles. Attributes, such as entity states, segment sensitivity, and access levels are evaluated to ensure alignment. For instance, higher risk states (*High Risk* or *Quarantined*) correspond to stricter access controls, enforced through meta-policies like `no_write_access_high_risk`.

Algorithm 2 formalizes the validation process, ensuring both semantic and risk-based compliance. The meta-policy language is formally structured with clearly defined syntax: each meta-policy comprises a name, type (any/all), and conditions. Conditions are expressed as logical predicates involving entity attributes, such as state, access level, and segment sensitivity.

In Algorithm 2, $\mathcal{M}$ represents the set of meta-policies, where each meta-policy m includes a condition ϕ_m that must evaluate to **True** for compliance. $\mathbf{x}$ is the feature vector extracted from the proposed policy p, containing relevant attributes, such as `state` or `access_level`. T denotes the decision tree constructed from $\mathcal{M}$, where each node n represents a condition ϕ_n for validation. $\mathbf{P}$ is the priority map used to resolve conflicts by assigning precedence to meta-policies. Violations is a list accumulating the meta-policies violated by the proposed policy, and ValidationResult indicates whether the policy complies (**True**) or fails (**False**). Logging

Algorithm 2 Precompiled Meta-Policy Validation

Require: $\mathcal{M}$ (Set of meta-policies), $\mathbf{x}$ (Feature vector), $\mathbf{P}$ (Priority map)
Ensure: ValidationResult, Violations
Violations $\leftarrow \emptyset$
Preprocess Meta-Policies:
Compile $\mathcal{M}$ into a decision tree T, where each node represents a condition ϕ_m from meta-policy m.
Feature Extraction:
Extract $\mathbf{x}$ from the proposed policy p.
Decision Tree Traversal:
for each node $n \in T$ **do**
    Evaluate ϕ_n with $\mathbf{x}$.
    if ϕ_n evaluates to **False then**
        Append n.meta-policy to Violations.
    end if
end for
Conflict Resolution:
Sort Violations by priority $\mathbf{P}$.
Resolve conflicts, retaining the highest-priority meta-policy per condition.
Validation Outcome:
if Violations $\neq \emptyset$ **then**
    ValidationResult $\leftarrow$ False
else
    ValidationResult $\leftarrow$ True
end if
return ValidationResult, Violations

mechanisms record violations and outcomes for auditing and debugging purposes.

Conflicts from overlapping meta-policy conditions are resolved systematically using a predefined priority map that ranks meta-policies according to organizational security objectives. In cases of conflict, the highest-priority meta-policy is enforced, ensuring consistent and predictable policy decisions.

D. Complexity Analysis

Algorithm 2 addresses scalability through precompilation of meta-policies into a decision tree, significantly reducing runtime validation complexity. This design ensures efficiency even as the number of meta-policies grows. Additionally, representing policies as compact feature vectors further minimizes computational overhead, ensuring high scalability and responsiveness in large-scale, dynamic environments. In the following, the complexity is discussed in more detail.

1) *Preprocessing Complexity:* Compiling meta-policies into a decision tree involves parsing and restructuring conditions. Let $|\mathcal{M}|$ be the number of meta-policies and a the average number of attributes per meta-policy. The preprocessing complexity is $O(|\mathcal{M}| \cdot a)$.
2) *Runtime Validation Complexity:* Traversing the decision tree has complexity proportional to its depth d, which depends logarithmically on the number of meta-policies in a balanced tree, i.e., $O(d)$ where $d \approx \log(|\mathcal{M}|)$. Extracting the feature vector $\mathbf{x}$ from the policy has complexity $O(k)$ where k is the number of relevant attributes in the policy. Thus, the total runtime complexity is $O(d+k) \approx O(\log(|\mathcal{M}|) + k)$.
3) *Conflict Resolution Complexity:* Sorting violations by priority involves a complexity of $O(v \cdot \log(v))$ where v is the number of violations. In the worst case, $v = |\mathcal{M}|$, leading to a complexity of $O(|\mathcal{M}| \cdot \log(|\mathcal{M}|))$.

The total complexity is finally to $O(|\mathcal{M}| \cdot a + \log(|\mathcal{M}|) + k + |\mathcal{M}| \cdot \log(|\mathcal{M}|))$.

E. Example

Let us consider a meta-policy `no_write_access_high_risk`, which enforces that entities in the *High Risk* state cannot have write access.

```
{
  "name": "no_write_access_high_risk",
  "type": "any",
  "condition": "state != 'High Risk'
  or access_level != 'full'"
}
```

In this policy, an entity with a `High Risk` must not have the `full access`, preventing it from having a write access.

Let us now consider a proposed policy for `iot_device1` that specificies:

```
{
  "entity_id": "iot_device1",
  "state": "High Risk",
  "access_level": "full"
}
```

To validate this specific policy against the meta-policy.

1) The feature vector is extracted: $\mathbf{x}$ = {state:'High Risk', access_level: 'full'}.
2) The decision tree evaluates $\phi =$ (state $\neq'$ *HighRisk*$'$) $\vee$ (access_level $\neq'$ *full*$'$). This evaluates to **False**.
3) The violation is logged: "`Policy violates meta-policy: no_write_access_high_risk`".
4) The algorithm returns `ValidationResult = False` and appends the violation to the `Violations` list.

This approach ensures that all proposed policies are rigorously validated against meta-policies, aligning with the principles of ZTA while optimizing resource efficiency.

VI. Proof of Concept Implementation: Qualitative Evaluation

To validate our ZTA framework, we implemented a Proof-of-Concept (PoC) network using a cluster of virtual machines (VMs) that simulate a controlled network environment. This setup executes key components of the ZTA system, including policy generation, enforcement, and risk assessment, in a small-scale network to analyze access scenarios and policy effectiveness.

Table III details the VMs used in this setup, outlining their primary functions, roles within the architecture, and specifications.

This PoC setup allows testing interactions between the various ZTA components, supporting simulated access and policy enforcement scenarios in a virtualized environment [35]. We qualitatively evaluate the benefits of the ZTA framework by

TABLE III
OVERVIEW OF VM SETUP FOR ZTA EXPERIMENT

Number	Primary Function	Role in Architecture	Specifications
VM1	Policy Administration (PA) and Policy Generator	Manages and updates security policies dynamically	4 CPU, 6 GB RAM, 12 GB disk
VM2	State Checking (PE)	Enforces policies based on device state	4 CPU, 6 GB RAM, 12 GB disk
VM3	Authenticator and Risk Assessment	Evaluates risk based on anomaly detection data	4 CPU, 4 GB RAM, 12 GB disk
VM4	Policy Enforcement Points (PEPs)	Executes updated security policies	4 CPU, 4 GB RAM, 12 GB disk
VM5	Server	Manages authentication status	1 CPU, 2 GB RAM, 12 GB disk
VM6	Simulated User Device	Simulates user access behavior	1 CPU, 2 GB RAM, 12 GB disk
VM7	Simulated IoT Device	Simulates IoT data transmission	1 CPU, 2 GB RAM, 12 GB disk
VM8	Simulated IoT Device	Simulates additional IoT data transmission	1 CPU, 2 GB RAM, 12 GB disk

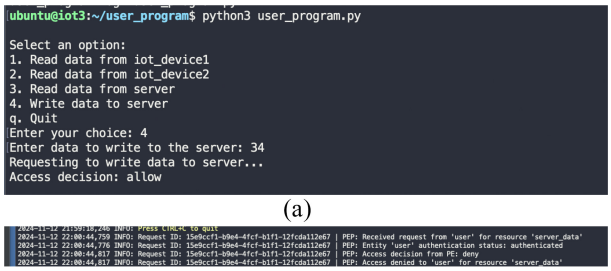

Fig. 5. Comparison of credential theft scenarios. (a) No ZTA, Allows attacker access if credentials are stolen. (b) Uses IP and context-aware rules to block unauthorized access attempts.

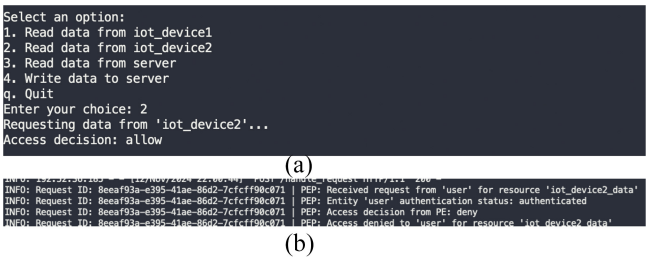

Fig. 6. Comparison of insider threat scenarios. (a) Without ZTA, allowing unauthorized access to sensitive devices. (b) Applies role- and context-based policies to limit access and block suspicious behavior.

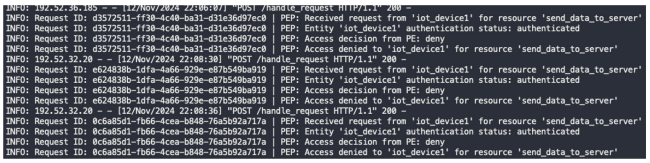

Fig. 7. ZTA reaction to DoS attack scenario.

analyzing the impact of possible attacks. We consider the following scenario.

1) The user has password-protected access to the server.
2) The user has read-only access to Device 1 but no access to Device 2.
3) Each IoT device can send data to the server at defined intervals.

A. Credential Theft and Unauthorized Server Access

Credential theft is a prevalent and critical attack vector where attackers exploit stolen user credentials to gain unauthorized access to sensitive systems. This scenario mimics real-world incidents like phishing attacks or brute-force credential compromises, which often bypass traditional password-based security systems. The lack of contextual verification mechanisms exacerbates this vulnerability, allowing attackers to leverage valid credentials undetected. Such breaches can lead to severe consequences, including unauthorized data exfiltration, system compromise, or further lateral movement within the network.

Without ZTA: The attacker can authenticate and gain full access to the server [Fig. 5(a)].

With ZTA: ZTA policies restrict access based on IP and user context, blocking unauthorized attempts from unfamiliar locations [Fig. 5(b)].

B. Insider Threat and Unauthorized Device Access

Insider threats pose significant risks to organizational security by exploiting legitimate access to systems. In this scenario, a user with authorized access to Device 1 attempts to gain unauthorized access to Device 2, bypassing conventional network controls. This scenario mimics real-world incidents where insiders abuse their permissions or credentials to access sensitive resources, a critical challenge for traditional security architectures. The lack of fine-grained, context-aware access control exacerbates the risk of exposing sensitive data or critical infrastructure to unauthorized users.

Without ZTA: Basic network security is bypassed, allowing access to Device 2 [Fig. 6(a)]. *With ZTA*: Role-based policies deny access to Device 2, and the PE flags any anomalous access attempts [Fig. 6(b)].

C. Compromised IoT Device and DoS Attack

In this scenario, an IoT device is compromised by an attacker and begins sending excessive amounts of data to the server, resulting in a Denial-of-Service (DoS) attack. This type of attack mimics real-world incidents where IoT devices, often lacking robust security controls, are exploited to overwhelm critical infrastructure. Such attacks are critical to address as they can cause server downtime, disrupt operations, and impact the availability of services.

Without ZTA: The server becomes overwhelmed, leading to potential downtime.

With ZTA: Rate-limiting policies prevent excessive data from any IoT device, mitigating the DoS attack (Fig. 7).

Fig. 8. ZTA reaction to suspicious user behavior.

D. Suspicious User Behavior and Anomaly Detection

This scenario involves a compromised user account engaging in actions that deviate significantly from its standard behavior patterns. Such behavior may include accessing sensitive resources, executing commands outside the user's typical scope, or initiating anomalous transactions. This scenario mimics real-world incidents where compromised credentials or insider threats exploit legitimate access to cause harm. Detecting these deviations is critical to preventing unauthorized activities and mitigating potential damage.

Without ZTA: Abnormal actions remain undetected, potentially causing damage.

With ZTA: Context-aware anomaly detection flags deviations, prompting the PE to require multifactor authentication or deny further access (Fig. 8).

VII. Quantitative Evaluation of Proposed ZTA

The PoC environment, detailed in Section VI, was configured to simulate typical access scenarios within an IIoT context under normal network operations. The implementation included critical ZTA components, such as the PA, PE, and PEP across a cluster of VMs.

It is important to clarify that our ZTA framework does not enforce policies at the granularity of individual packets traversing the network. Instead, policy enforcement occurs primarily at session initiation and at strategic checkpoints or upon detecting anomalous behavior. Specifically, a lightweight probe passively inspects network traffic without directly interfering with routing, collecting contextual information and real-time threat indicators. Based on this information, the PEP enforces security policies proactively at connection setup and reactively when anomalies are detected. This approach minimizes computational overhead and maintains high security standards without unnecessarily impacting network performance.

The evaluation aimed to measure performance metrics, including latency, CPU usage, and memory utilization, during normal network operations. These metrics provide insights into the baseline performance of the ZTA framework, demonstrating its efficiency and scalability in regular conditions.

A. Latency

Fig. 9 shows the comparison of latency with and without ZTA across different request rates. Latency was measured as the average time required to process access requests under varying network loads. During normal operations, latency remained within acceptable bounds. Without ZTA, latency was consistently low and averaged approximately 45 ms across a range of request rates. With ZTA, latency increased due to context validation and policy enforcement checks but remained well below the threshold for near-real-time IIoT operation. The average latency under ZTA peaked at approximately 141 ms

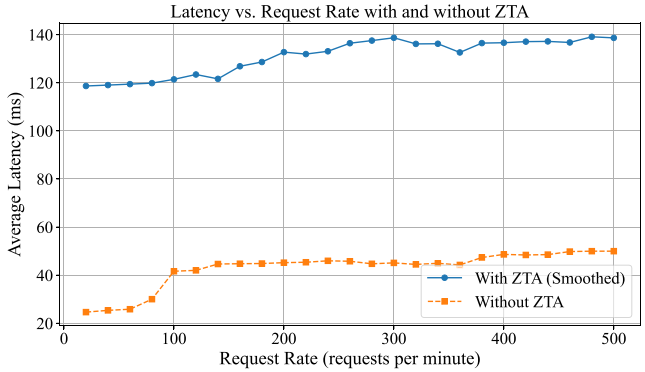

Fig. 9. Latency versus Request Rate with and without ZTA.

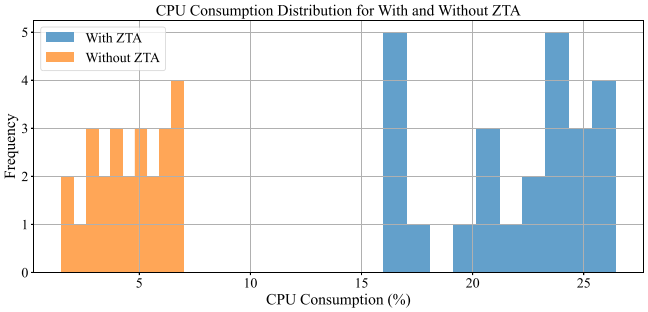

Fig. 10. CPU scaling with/without ZTA.

at the highest simulated traffic levels. This controlled rise in latency demonstrates the effectiveness of the ZTA framework in maintaining operational responsiveness under increasing workload.

B. CPU and Memory Utilization

Fig. 10 illustrates CPU consumption relative to the request rate, demonstrating consistent and predictable scaling as traffic volume increases. To assess the computational efficiency of the ZTA framework under normal operation, CPU usage was recorded across all major components of the PoC environment.

In deployments without ZTA, CPU utilization ranged from 1.5% to 7.0%, reflecting minimal computational burden associated with basic access control mechanisms. In contrast, the ZTA deployment began at approximately 16% and scaled linearly with traffic, reaching a maximum of 26.4% at higher request rates. This increase is primarily attributable to the additional workload introduced by real-time risk assessment, policy lookup, and state-checking logic executed within the PEP and PE.

To further clarify the distribution of load across system components, we conducted per-VM profiling under typical traffic conditions. The observed 26.4% peak represents a balanced workload across all CPU cores, avoiding saturation on any single thread. Average CPU utilization per VM was as follows.

1) *VM2 (PE and State Checking):* $25.1\% \pm 3.8\%$.
2) *VM3 (Authenticator and Risk Assessment):* $21.3\% \pm 4.0\%$.
3) *VM4 (PEPs - PEPs):* $26.4\% \pm 5.1\%$.

4) *VM5 (Server):* $12.2\% \pm 2.3\%$.
5) *VM6 (Simulated User Device):* $8.5\% \pm 1.4\%$.
6) *VM7 and VM8 (Simulated IoT Devices):* $6.8\% \pm 1.1\%$.

The VMs simulating IoT endpoints (VM7 and VM8) executed lightweight scripts that periodically generated and transmitted synthetic telemetry to the server, simulating typical IIoT sensor communication patterns. This emulation enabled us to benchmark system behavior under realistic edge-to-core traffic flows without introducing unnecessary complexity at this stage. In future work, we intend to incorporate more representative IIoT simulators and heterogeneous device types to further refine performance profiling across protocol stacks.

Memory usage remained stable throughout all test runs. Core ZTA components particularly the PE and PEP used modest memory overhead to retain per-session context and temporary policy caches. Overall memory consumption remained within the 22–24% range across core components, indicating effective resource allocation. These results confirm that the proposed framework provides scalable and context-aware policy enforcement without imposing excessive resource demands, making it suitable for deployment on mid-tier edge gateways and IIoT control nodes.

C. Performance Metrics and Threat Response

To comprehensively evaluate the operational overhead and dynamic threat response capabilities of the proposed ZTA, we conducted a detailed performance analysis across three distinct scenarios: 1) *Normal Operation*, representing standard conditions; 2) *High-Frequency Access*, simulating elevated request rates; and 3) a targeted *DoS Attack*, simulating adversarial behavior. We measured key performance metrics (CPU usage, memory consumption, latency, network throughput, and packet loss) across critical system components: User, Server, PEP, PE.

Table IV summarizes the average performance metrics, while Figs. 11–13 visually depict the system's behavior under each scenario.

Under Normal Operation, the ZTA framework exhibited stable and efficient resource management. As shown in Fig. 11, CPU usage remained modest across components, averaging below 15%. Memory consumption followed a similar trend, as illustrated in Fig. 12, remaining around 22% on average. Network latency, was low 120.5 ms for the User and 130.2 ms for the PEP while Fig. 13 shows throughput was high (42.8 Mb/s for User and Server), with minimal packet loss (2.1% for User, 3.4% for PEP). These results confirm that the system imposes minimal overhead during standard IIoT operations.

In the High-Frequency Access scenario, the system experienced moderate stress due to increased request rates. CPU usage increased noticeably (see Fig. 11), reaching 22.3% for the User and approximately 20% for the PEP. Fig. 12 shows memory usage rose to nearly 28.6% (User) and 26.5% (PEP). Latency increased by about one-third, with Table IV over latency showing a jump to 160.7 ms (User) and 175.9 ms (PEP). As seen in Fig. 13, throughput slightly declined (8%), and packet loss increased to 8.5% and 9.8% respectively. Despite these increases, system performance remained within operational limits.

TABLE IV
PERFORMANCE METRICS ACROSS SCENARIOS

Metric	Entity	Normal	HighFreq	DoS
CPU Usage (%)	User	14.2	22.3	27.5
	Server	11.8	18.5	19.7
	PEP	13.1	20.1	22.4
	PE	12.7	19.2	21.1
Memory Usage (%)	User	23.5	28.6	31.2
	Server	21.1	24.3	26.4
	PEP	22.8	26.5	28.1
	PE	21.7	25.8	27.9
Throughput (Mbps)	User	42.8	39.2	35.8
	Server	42.8	39.2	35.8
	PEP	41.7	37.5	34.6
Latency (ms)	User	120.5	160.7	210.4
	PEP	130.2	175.9	248.9
Packet Loss (%)	User	2.1	8.5	16.5
	PEP	3.4	9.8	18.3

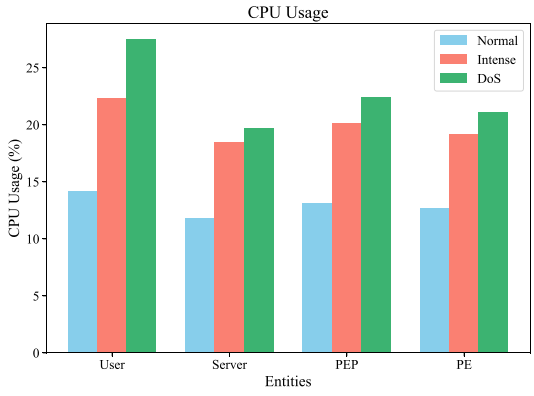

Fig. 11. CPU usage across scenarios.

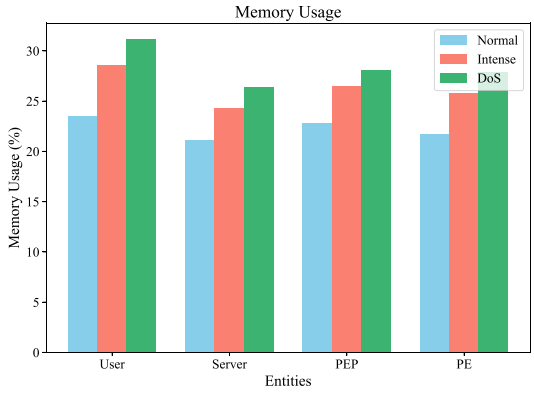

Fig. 12. Memory usage across scenarios.

During the simulated DoS Attack, the attacker generated traffic exceeding 30 requests per 10 s, overwhelming normal thresholds. The PEP's anomaly detection module promptly identified this pattern as a data flooding attack with 95% confidence. The resulting alert triggered a state transition in the FSM to High Risk, activating strict access controls via the Risk Assessment module.

The attack's impact on performance was substantial: as shown in Fig. 11, CPU usage surged to 27.5% (User) and 22.4% (PEP). Memory usage also spiked (Fig. 12), reaching 31.2% (User) and 28.1% (PEP). Latency, illustrated in Table IV over latency, escalated sharply to 210.4 ms (User) and 248.9 ms (PEP). Throughput, as shown in Fig. 13,

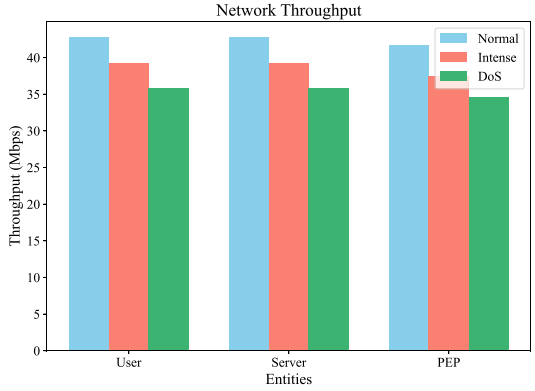

Fig. 13. Network throughput across scenarios.

dropped to 35.8 Mb/s (User and Server), and packet loss rose to 16.5% (User) and 18.3% (PEP). Despite this degradation, the Server and PE sustained stable performance, indicating effective containment of the compromised entity.

These comprehensive results confirm the ZTA framework's capacity for real-time threat detection, dynamic policy enforcement, and resilience under both elevated workload and active adversarial conditions. The architecture isolates threats without compromising the stability of the broader system.

D. Scalability, Interoperability, and Edge Deployment Considerations

To further enhance the practical deployment of the proposed ZTA framework in IIoT environments, several key considerations have been identified for future work.

1) Edge Device Deployment: Resource constraints, such as limited CPU, memory, and energy availability, are critical challenges for IIoT edge devices. While our PoC focused on server-grade VMs, future work will involve implementing lightweight PEPs and context probes on resource-constrained devices, such as Raspberry Pi and microcontrollers. Optimization strategies, including offloading intensive computations to nearby edge servers and minimizing local policy enforcement overhead, will be investigated to ensure scalability to highly distributed, resource-limited environments.

2) Multitenancy Support: In industrial ecosystems, multitenancy is often required where multiple independent organizations or operational domains coexist. Scaling the ZTA framework to support multiple tenants will involve isolated policy domains, federated risk assessments, and tenant-aware PEPs/PEs to ensure secure, logical separation while sharing underlying infrastructure.

3) Protocol Interoperability: Although our current implementation operates over standard TCP/IP, real-world IIoT systems rely heavily on specific industrial protocols, such as MQTT and OPC UA. Future iterations of the framework will integrate protocol adapters and semantic translators to support these standards natively, allowing for dynamic, context-aware security policies across heterogeneous communication stacks.

4) Scale-Out of PEPs and PEs: Supporting large-scale IIoT deployments requires scalable PEP and PE architectures. Future designs will employ hierarchical or distributed control models to allow PEPs and PEs to scale horizontally, with mechanisms for distributed policy synchronization, context aggregation, and decentralized anomaly detection to maintain performance and resilience across large, dynamic networks.

VIII. Conclusion

In this article, we proposed a dynamic ZTA framework tailored specifically for IIoT environments. Our solution integrates real-time contextual data, anomaly detection, adaptive risk management, and dynamic policy enforcement, directly addressing the limitations of traditional static approaches. We demonstrated through a practical PoC evaluation that our ZTA framework effectively mitigates common cybersecurity threats, such as credential theft, insider threats, compromised IIoT devices, and anomalous user behaviors, without introducing significant latency or computational overhead. Quantitative evaluations indicated that the proposed architecture achieves acceptable latency and balanced CPU usage across multiple VMs. Specifically, policy enforcement at strategic checkpoints rather than per-packet enforcement ensures computational efficiency while preserving robust security. Moreover, statistical analysis of resource consumption demonstrated good scalability by exploiting multicore processing capabilities.

While the current evaluation employed an emulated IIoT device to demonstrate typical data transmission scenarios, future work will study the impact of the architecture. Indeed, IIoT devices often have limited energy, memory, and processing power, making the implementation of sophisticated countermeasures particularly challenging. Future research will empirically evaluate the framework on resource-constrained hardware platforms (e.g., Raspberry Pi) to confirm its practical viability and performance scalability. Relying on edge devices to implement some part of the architecture may help the network improve its scalability. Besides, we expect to align our framework with common industrial standards, such as OPC UA and MQTT to ensure seamless integration into practical industrial settings. Additionally, we plan to perform a formal sensitivity analysis of the FSM transition thresholds to evaluate their influence on detection accuracy and false-positive rates, and to explore adaptive calibration mechanisms guided by runtime feedback and operational context.

References

[1] M. Asiri, N. Saxena, R. Gjomemo, and P. Burnap, "Understanding indicators of compromise against cyber-attacks in industrial control systems: A security perspective," *ACM Trans. Cyber-Phys. Syst.*, vol. 7, no. 2, pp. 1–33, 2023.

[2] V. Stafford, "Zero trust architecture," NIST, Gaithersburg, MD, USA, document SP 800-207, 2020.

[3] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.

[4] Q. Yao, Q. Wang, X. Zhang, and J. Fei, "Dynamic access control and authorization system based on zero-trust architecture," in *Proc. 1st Int. Conf. Control, Robot. Intell. Syst.*, 2020, pp. 123–127.

[5] X. Feng and S. Hu, "Cyber-physical zero trust architecture for industrial cyber-physical systems," *IEEE Trans. Ind. Cyber-Phys. Syst.*, vol. 1, pp. 394–405, 2023.

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Nist special publication 800-207 zero trust architecture," U.S. Dept. Commerce, NIST, Gaithersburg, MD, USA, document SP 800-207, 2020.
[7] J. Kindervag et al., *Build Security Into Your Network's DNA: The Zero Trust Network Architecture*, vol. 27, Forrester Res. Inc., Cambridge, MA, USA, pp. 1–16, 2010.
[8] E. Gilman, *Zero Trust Networks: Building Systems in Untrusted Networks*, O'Reilly, Sebastopol, CA, USA, 2016.
[9] X. Yan and H. Wang, "Survey on zero-trust network security," in *Proc. Int. Conf. Artif. Intell. Security (ICAIS)*, Hohhot, China, Jul. 2020, pp. 50–60.
[10] E. Bertino, "Zero trust architecture: does it help?" *IEEE Security Privacy*, vol. 19, no. 5, pp. 95–96, Sep./Oct. 2021.
[11] B. Osborn, J. McWilliams, B. Beyer, and M. Saltonstall, "BeyondCorp: Design to deployment at Google," *USENIX*, vol. 41, no. 1, pp. 28–34, 2016.
[12] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Security*, vol. 4, no. 3, pp. 224–274, 2001.
[13] E. B. Fernandez and A. Brazhuk, "A critical analysis of zero trust architecture (ZTA)," *Comput. Stand. Interfaces*, vol. 89, Apr. 2024, Art. no. 103832.
[14] M. Howlett and M. Ramesh, "Designing for adaptation: Static and dynamic robustness in policy-making," *Public Admin.*, vol. 101, no. 1, pp. 23–35, 2023.
[15] L. Bradatsch, O. Miroshkin, and F. Kargl, "ZTSFC: A service function chaining-enabled zero trust architecture," *IEEE Access*, vol. 11, pp. 125307–125327, 2023.
[16] S. Hong, L. Xu, J. Huang, H. Li, H. Hu, and G. Gu, "Sysflow: Toward a programmable zero trust framework for system security," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2794–2809, 2023.
[17] Z. Niu, L. Dong, and Y. Zhu, "The runtime model checking method for zero trust security policy," in *Proc. ICCSIE*, 2022, pp. 8–12.
[18] Z. Zaheer, H. Chang, S. Mukherjee, and J. Van der Merwe, "eZTrust: Network-independent zero-trust perimeterization for microservices," in *Proc. Symp. SDN Res. (SOSR)*, 2019, pp. 49–61.
[19] B. Paul and M. Rao, "Zero-trust model for smart manufacturing industry," *Appl. Sci.*, vol. 13, no. 1, p. 221, 2022.
[20] C. Zanasi, S. Russo, and M. Colajanni, "Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures," *Ad Hoc Netw.*, vol. 156, Apr. 2024, Art. no. 103414.
[21] S. Xiao, Y. Ye, N. Kanwal, T. Newe, and B. Lee, "SoK: context and risk aware access control for zero trust systems," *Security Commun. Netw.*, vol. 2022, no. 1, 2022, Art. no. 7026779.
[22] W. Iqbal, H. Abbas, M. Daneshmand, B. Rauf, and Y. A. Bangash, "An in-depth analysis of IoT security requirements, challenges, and their countermeasures via software-defined security," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10250–10276, Oct. 2020.
[23] K. S. Sahoo and D. Puthal, "SDN-assisted DDoS defense framework for the Internet of Multimedia Things," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 16, no. 3S, pp. 1–18, Dec. 2020.
[24] J. Bhayo, S. Hameed, and S. A. Shah, "An efficient counter-based DDoS attack detection framework leveraging software defined IoT (SD-IoT)," *IEEE Access*, vol. 8, pp. 221612–221631, 2020.
[25] N. Saha, S. Misra, and S. Bera, "Q-Flag: QoS-aware flow-rule aggregation in software-defined IoT networks," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 4899–4906, Apr. 2022.
[26] S. H. A. Kazmi, R. Hassan, F. Qamar, K. Nisar, and M. A. Al-Betar, "Federated conditional variational auto encoders for cyber threat intelligence: Tackling non-iid data in SDN environments," *IEEE Access*, vol. 13, pp. 26273–26288, 2025.
[27] A. Montazerolghaem, "Software-defined Internet of Multimedia Things: Energy-efficient and load-balanced resource management," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 2432–2442, Feb. 2022.
[28] T. Zhang et al., "How to mitigate DDoS intelligently in SD-IoV: A moving target defense approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1097–1106, Jan. 2023.
[29] F. Stodt, F. Theoleyre, and C. Reich, "Advancing network survivability and reliability: Integrating XAI-enhanced autoencoders and LDA for effective detection of unknown attacks," in *Proc. DRCN*, 2024, pp. 9–16.
[30] F. Stodt, C. Reich, and F. Theoleyre, "Context-aware anomaly detection by community detection in the Internet of Things," unpublished.
[31] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK®: Design and philosophy," MITRE, Bedford, MA, USA, Rep. MP180360R1, 2020.
[32] "Guide for conducting risk assessments," Joint Task Force Transformation Initiative, NIST, Gaithersburg, MD, USA, Rep. SP 800-30 Rev. 1, Sep. 2012. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/30/r1/final
[33] (OWASP Found., Wakefield, MA, USA). *OWASP Risk Rating Methodology*. (2025). [Online]. Available: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology
[34] Y. Wu, B. Sicard, and S. A. Gadsden, "Physics-informed machine learning: A comprehensive review on applications in anomaly detection and condition monitoring," *Expert Syst. Appl.*, vol. 255, Dec. 2024, Art. no. 124678.
[35] F. Stodt, "ZTA," GitHub Repository, 2024. Accessed: Nov. 19, 2024. [Online]. Available: https://github.com/f11691/ZTA2

Fatemeh Stodt (Student Member, IEEE) is currently pursuing the joint Ph.D. degree with the University of Strasbourg, Strasbourg, France, and Furtwangen University, Furtwangen im Schwarzwald, Germany.

She is a Researcher with the University of Strasbourg and Furtwangen University, focusing on blockchain technology, cybersecurity, and industrial Internet of Things.

Christoph Reich received the Ph.D. degree in computer science from DeMontfort University, Leicester, U.K., in 1999.

He is a Professor with the Furtwangen University, Furtwangen, Germany, and the Head of the Institute of Data Science, Cloud Computing, and IT Security. He has authored more than 180 research papers. His research interests are machine learning, distributed systems, and cybersecurity.

Fabrice Theoleyre (Senior Member, IEEE) received the Ph.D. degree in computer science from INSA Lyon, Villeurbanne, France, in 2006.

He is a Senior Researcher with the CNRS, Strasbourg, France. After spending two years with the Grenoble Informatics Laboratory, Saint-Martin-d'Hères, France, he joined the ICube laboratory in Strasbourg, Strasbourg, where he has been working since 2009. Over the years, he has held several visiting research positions, including with the University of Waterloo, Waterloo, ON, Canada, Inje University, Gimhae, South Korea, and Jiaotong University, Shanghai, China. His research focuses primarily on distributed algorithms, low-power protocols, and resiliency of wireless networks.